

Raspberry Pi Pico LED Controller Implementation

The implementation of all this on a Raspberry Pi Pico in order to control the LEDs is fairly straightforward.

What you will need:

- GPIO Pins (2x)
- 3V3_OUT
- GND

Wiring:

- Pico GPIO #1 to RF board DATA*
- Pico GPIO #2 to RF board CLOCK*
- Pico 3V3_OUT to the RF board VCC
- Pico GND to RF board GND

*Note: From here on DATA and CLOCK will refer to the respective pin for the applicable RF board in use.

Argon:

DATA = ARGON_DATA
CLOCK = ARGON_CLK

Boron:

DATA = BORONFPM_DATA
CLOCK = BORONFPM_CLK

Pico Implementation:

To begin with, the following is assuming you are providing power to the RF board via Raspberry Pi Pico. It should be noted that the RF board can additionally power the pico when plugged into the Xbox 360 console, and behavior while receiving commands from the console will need to be factored into your implementation.

With the wiring in place, when the Raspberry Pi Pico receives power the 3V3_OUT pin will power the RF board. After initializing the GPIO pins for DATA (GPIO OUT) and CLOCK (GPIO IN) the first thing we need to do is pull up each respective pin. This is especially important because the CLOCK will not begin to cycle unless told to do so. When the RF board is plugged into the Xbox 360 console this is typically handled by the SMC, however we need to send the signals ourselves when the RF board is not receiving signals from the Xbox 360. Now that the GPIO pins are initialized properly the RF board should be in a state ready to accept commands with CLOCK cycling between LOW and HIGH.

While the commands we will be sending are 9-bit, the acknowledgment bit preceding the command to signify where the data comes from will be 0 (SMC → FPM) for LED control commands. Additionally the execution of this bit is on the same timing as the 9-bit command execution. As such, we can consider these commands to be 10-bit rather than 9-bit when storing the data for sending commands with our pico.

Before we can send the first command bit we need to signal to the RF board that a command is about to be sent. This is done by first setting the DATA pin LOW. We can now send the first bit of the command through the DATA pin. It should be noted that, especially with Boron RF boards, the clock phase timing

is very important when sending command data. Each bit needs to be sent on the LOW side of the CLOCK cycle, which consists of one LOW and one HIGH phase. Using the CLOCK phase to time sending the data we can continue to send each consecutive bit of the command on the LOW side of the clock until we reach the end of the command. For the final command bit, once the proceeding HIGH phase of the clock's cycle has been complete we are ready to signal the end of the command to the RF board. To do this we wait one additional full cycle of the clock and then send HIGH through the DATA pin when CLOCK is HIGH. If a full cycle is not given between the final command bit cycle and the end-of-command signifying bit the pico will be unable to execute consecutive commands properly. This sums up the detailed implementation of an LED controller for RF boards using a Raspberry Pie Pico.

Abstract:

1. Initialize pico GPIO pins for DATA (out) and CLOCK (in).
2. Pull up CLOCK and DATA pin.
3. Set DATA pin LOW to prepare RF board to receive command.
4. Send each consecutive bit of the 10-bit* command sequence on LOW phase of clock.
5. After the final command bit's proceeding HIGH phase of CLOCK is complete, wait one full CLOCK cycle.
6. Set DATA pin HIGH while CLOCK is HIGH to signal to RF board end of command sequence.

*Note: Commands are actually 9-bit with a preceding acknowledgment bit. See Pico Implementation section for detailed explanation.